



دانشگاه کیدان

تمرین سری دوم

درس پایگاه داده - دکتر برشبان

1404-1

بخش اول – سوالات تئوری

سوال 1. مرکز تعمیرات موبایل "موبوگیلان" به دنبال تغییر در تیم فنی خود است. کارشناس پایگاه داده قبلی به دلیل اختلاف با مدیران، پشتیبانی را متوقف کرده و با درخواست افزایش حقوق بالا، کل کسب و کار را مختل کرده و در خطر قرار داده است. مدیر مجموعه تصمیم دارد همکاری را قطع کند و به جای او یک کارشناس جدید استخدام کند، اما شرط استخدام این است که دانش و فرآیندهای موجود کاملاً مستندسازی شده تحویل داده شوند.

شما به عنوان کارشناس جدید استخدام شده اید. در ابتدای کار از آنجا که کارشناس قبلی هیچ مستندی از پیاده سازی و عملکرد پایگاه داده ایجاد نکرده شما تنها به کوئری ها دسترسی دارید. در قدم اول باید این کوئری ها را بخوانید، رفتار آن ها را متوجه شوید و مستندسازی کنید.

بخش الف) کوئری زیر را در نظر بگیرید. (30 امتیاز)

```
SELECT r.ID AS RepairID, r.Issue, r.EstimatedPrice, r.FinalPrice, r.Details, r.DateIn,
r.DateOut, u.ID AS UserID, u.NationalID, u.FullName, u.MobilePhone,
u.LandlinePhone, u.Referral, d.ID AS DeviceID, d.IMEI, d.Color, pm.ModelName
FROM Repairs r
JOIN Users u ON r.UserID = u.ID
JOIN RegisteredDevices d ON r.DeviceID = d.ID
JOIN PhoneModels pm ON d.ModelID = pm.ID
WHERE r.AdminID = ? AND ${whereClause}
ORDER BY r.DateIn DESC
```

توضیح: در کوئری بالا ؟ یک پارامتر زمان اجرا (مثلاً شناسه ادمین میزبان) و \${whereClause} یک تکه شرطی است که توسط اپلیکیشن تولید می شود (مثلاً جستجو یا فیلترها).

از 5 سوال زیر فقط به 3 سوال به انتخاب خود پاسخ دهید.

1. هر جدول (Repairs, Users, RegisteredDevices, PhoneModels) چه اطلاعاتی به مجموعه نتایج می آورد؟

2. علت استفاده از JOIN در این کوئری چیست؟ تفاوت رفتار خروجی این کوئری با استفاده از LEFT JOIN در این مکان چه خواهد بود؟
3. نقش شرط `AdminID = ?` را توضیح دهید؛ این پارامتر در چه سناریوهایی ضروری است؟
4. `ORDER BY r.DateIn DESC` چه تأثیری در نحوه نشان دادن داده‌ها دارد؟ یک دلیل عملی برای انتخاب این ترتیب در داشبورد تعمیرات بنویسید.
5. `{whereClause}` توسط اپلیکیشن ساخته می‌شود. دو یا سه دسته منطقی از فیلتر/شرط را که ممکن است در آن قرار گیرد شرح دهید.

(بخش ب) کوئری زیر را در نظر بگیرید. (30 امتیاز)

```
SELECT pc.ID AS CategoryID, pc.CategoryName, pl.ID AS PriceListID, pl.Title,
pl.ShowInFardPlus
FROM PhoneCategories pc
LEFT JOIN PriceLists pl ON pl.CategoryID = pc.ID AND pl.AdminID = ?
LEFT JOIN PriceListServices pls ON pls.PriceListID = pl.ID
LEFT JOIN Services s ON s.ID = pls.ServiceID AND s.AdminID = pl.AdminID
LEFT JOIN PhoneModels pm ON pm.CategoryID = pc.ID
LEFT JOIN PriceValues pv ON pv.PriceListID = pl.ID AND pv.ModelID = pm.ID AND
pv.ServiceID = s.ID
```

از 5 سوال زیر فقط به 3 سوال به انتخاب خود پاسخ دهید.

1. این کوئری سعی دارد چه چیزی را نشان دهد.
2. چرا در این کوئری از چند LEFT JOIN استفاده شده است؟
3. شرط `pl.AdminID = ?` درون شرط LEFT JOIN آمده است. اگر همین شرط را به WHERE منتقل کنیم چه تفاوتی در خروجی به وجود می‌آید و چرا طراح این کوئری آن را در JOIN قرار داده است؟
4. چرا در ردیف (pv) PriceValues هر سه کلید (PriceListID, ModelID, ServiceID) لازم است؟
5. فرض کنید یک دسته PhoneCategory مدلهایی دارد ولی برای ادمین جاری هیچ PriceList تعریف نشده است. این کوئری برای آن دسته چه ردیف‌هایی تولید می‌کند و چه مقادیری در ستون‌های PriceListID، ServiceName و Price می‌بینیم؟

بخش ج) (10 امتیاز)

فرض کنید در سمت کلاینت، کاربران ورودی یک فیلد را به شکل زیر می‌تواند وارد نمایند:

Please Enter Your User ID:

و در سمت سرور کوئری به شکل زیر ایجاد میگردد:

```
var query = "SELECT * Users WHERE id = " + input;
```

شما به عنوان یک کارشناس در مرحله‌ی اول باید متوجه شوید که روش بالا چه مشکلی ایجاد می‌نماید. بنظرتان مشکل چیست؟

سوال 2. می‌خواهیم یک سامانه مدیریت فعالیت‌های کشاورزی برای یک شرکت طراحی کنیم (مشابه تمرین 1، سوال 3).

شرح کلی:

- شرکت دارای چند مزرعه (Farm) است.
- در هر مزرعه چند محصول (Crop) کاشته می‌شود.
- برای هر محصول، مجموعه‌ای از فعالیت‌ها (Activity) مانند آبیاری، کوددهی، برداشت و ... انجام می‌شود.
- کارگران (Employee) مسئول انجام این فعالیت‌ها هستند.
- ممکن است یک فعالیت توسط چند کارگر انجام شود.
- رابطه‌ی بین کارگر، محصول و فعالیت شامل اطلاعاتی مثل مدت کار (hours_worked) و دستمزد پرداختی (payment) است (راهنمایی: برای ذخیره این اطلاعات می‌توانید یک جدول با عنوان work_records در نظر بگیرید).

خواسته‌ها:

1. مدل‌سازی منطقی انجام دهید. (20 امتیاز)

2. دستورات SQL برای ایجاد جداول را بنویسید. (10 امتیاز)

بخش دوم – سوالات عملی

قبل از پاسخ به بخش سوالات عملی لازم است به نکات زیر توجه فرمائید:

1. کد دانشجوئی خود را حتما بصورت صحیح وارد نمائید.
 2. در سوالات دستورات به گونه ای اجرا می شوند که پایگاه داده را تغییر می دهند، پس حتما سوالات را به ترتیب ایجاد شده اجرا نمائید که مشکلی ایجاد نشود.
 3. در آخر لطفا کد Generate شده را دستکاری نکنید.
- فایل های مربوط به تمرین عملی را می توانید از این [لینک](#) دریافت نمائید؛ توضیحات مرتبط با تمرین عملی نیز در انتهای همین فایل برای شما قرار گرفته است.

دستورالعمل ها:

- به تمامی سؤالات به صورت واضح پاسخ دهید.
- تمرین تنها به صورت انفرادی قابل انجام بوده و در صورت وجود هرگونه تشابه، **نمره صفر** لحاظ میشود.
- از پاسخ های دست نویس تا حد امکان خودداری کنید.
- پاسخ را در قالب یک فایل pdf با نام DB_HW02_StudentNumber_FirstnameLastname.pdf در کوئرا آپلود فرمائید.

موفق باشید

پایگاه داده 14041

دکتر برشبان

دستیاران آموزشی: محمدمتین رضائی فرد – بهراز فرشته صنیعی

SQL Practice Assignment - Northwind Database

Overview

This assignment provides a set of **SQL query challenges** based on the **Northwind** database.

Each student will automatically receive a **set of questions** generated based on their **student number**.

Your task is to **write SQL queries** that correctly answer each question, execute them using Python, and **submit your work by uploading a zip file containing the `Student_Number.txt` and the resulting JSON file (`query_hashes.json`)**.

Project Structure

```
project/
├──
├── databases/
│   ├── northwind.db      ← Main database for SQL queries
│   └── questions.db      ← Contains questions
├──
├── config/
│   └── Student_Number.txt ← Your student number (used to generate questions)
├──
├── questions/
│   ├── Question_0.py     ← Auto-generated question files (you will fill in answers here)
│   ├── Question_1.py
│   └── ...
├──
├── answer/
│   └── query_hashes.json  ← Auto-saved results of your answers (do not delete)
├──
└── generate_questions.py  ← The main generator script
```

Step 1 - Setup

1. Make sure you have **Python 3.10+** installed.
2. Install **Rich** for colored console output:

```
pip install rich
```

3. Verify that the following folders exist:

```
./databases
./config
./questions
./answer
```

4. Inside `./config`, make sure the file `Student_Number.txt` contains *only your student number*, for example:

```
1401012345678
```

Step 2 - Generate Your Questions

Run the `generate_questions.py`:

This will:

- Read your student number.
- select 12 questions for you.
- Create files named `Question_0.py`, `Question_1.py`, ..., in the `./questions` folder.

Each file will already include:

- The question text.

- The code to connect to the Northwind database.
- Code to save your answer hash automatically.

Step 3 - Answer Each Question

1. Open each file (e.g., `questions/Question_0.py`) in your editor.
2. Find the section:

```
# --- Fill in your SQL query here ---
answer = ''

'''
# -----
```

3. Write your SQL query **inside** the triple quotes.
Example:

```
answer = '''
SELECT * FROM Employees;
'''
```

4. Save the file and run it
5. The output will:
 - Print your query results in a **Rich table**.
 - Automatically store a hash of your result in `answer/query_hashes.json`.

Step 4 - Verify Your Answers

You can check your progress in `answer/query_hashes.json`.
Each entry will look like this:

```
{
  "Q0": {
    "answer": "SELECT * FROM Employees;",
    "hash": "7e5b62f84a1e5..."
  },
  "Q1": {
    "answer": "...",
    "hash": "..."
  }
}
```

If `"hash"` is `"Failed"`, it means your query didn't run correctly (syntax or runtime error).

Step 5 - Submit

Submit only a zip file containing the following files/folders:

```
/config/Student_Number.txt
/answer/query_hashes.json
```

Do **not** include the database files or generator script.

Notes

- Do **not** modify anything outside the triple-quoted SQL area.
- Do **not** rename files or move them.
- Each question file must run successfully without errors.
- Your final `query_hashes.json` must contain 12 valid hashes.

Troubleshooting

Problem	Solution
<code>sqlite3.OperationalError: no such table</code>	Ensure <code>.db</code> files exist in <code>./databases</code>
<code>no such function: YEAR</code>	SQLite doesn't support <code>YEAR()</code> . Use <code>strftime('%Y', OrderDate)</code> instead.
<code>adjust the *_PATH variables at the beginning of each file to fix file path related errors</code>	